ZERO-CLOUD // ZERO-TRUST // AIR-GAPPED

RED STRING

The Complete Beginner's Guidebook

A crystal-clear, step-by-step architectural breakdown of the peer-to-peer data transport fabric & edge intrusion detection engine built on IBM LinuxONE mainframe technology.

0.6 ms	< 2 Frames	0 Bytes
EDGE ML LATENCY	QUARANTINE SPEED	CLOUD EXFILTRATION

TARGET SYSTEM: IBM LINUXONE (S390X) • RED HAT ENTERPRISE LINUX 9.8

1. The Big Picture (The 5-Year-Old Explanation)

The Tin Can Analogy:

Imagine two kids talking through two tin cans connected by a **tight Red String** across their backyards. There is NO cloud server, NO internet middleman, and NO company listening. Just a direct string. If a bad guy sneaks onto the string to steal your secrets, our smart bouncer cuts the string with scissors in less than 2 milliseconds!

Meet the 3 Characters:



Good Monkey

The Human User

Types at normal human speed (~150ms between keys), copies normal English words, and drinks coffee. Always allowed!



Evil Robot

Malware / Scraping Bot

Tries to vomit 2,500 encrypted files per second through the string to steal data before anyone notices.



The Bouncer

Edge ML (Isolation Forest)

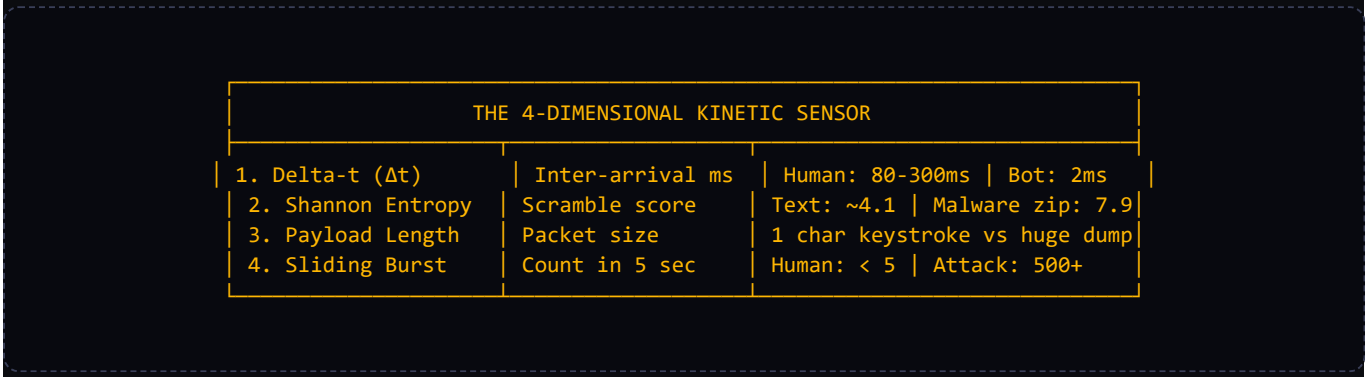
Stands on the string. Does NOT read your private messages. Only watches **the rhythm and shake of the string**. If it shakes like a robot, it cuts the cord instantly!

How It Is Different from Regular Apps:

FEATURE	NORMAL APPS (SLACK, CLOUD SYNC)	RED STRING ZERO-TRUST
Where Data Travels	Sent to a cloud server in California	**Direct Local Socket (0.0.0.0:7788)**
Privacy Inspection	Scans cleartext keywords on their servers	**Zero Cleartext Payload Reading**
Malware Defense	Reacts minutes or days later	**Severs in < 2 frames (~0.6ms)**
Audit Trail	Modifiable database logs	**Cryptographic SHA-256 Hash Chain**

2. Sniff the Movement, Never Read the Secret

Most security systems spy on you by reading your private passwords, letters, and clipboard contents. **Red String is completely blind to cleartext.** It looks exclusively at the physics (kinetics) of packet arrival.



The 4 Super-Sensors Explained:

1. **Gap Time (Δt in milliseconds)**
   Humans take at least 80ms to press the next key. Malware scripts fire packets in 0.5ms to 2ms. If packets arrive at machine-gun speed, an alert flag rises.

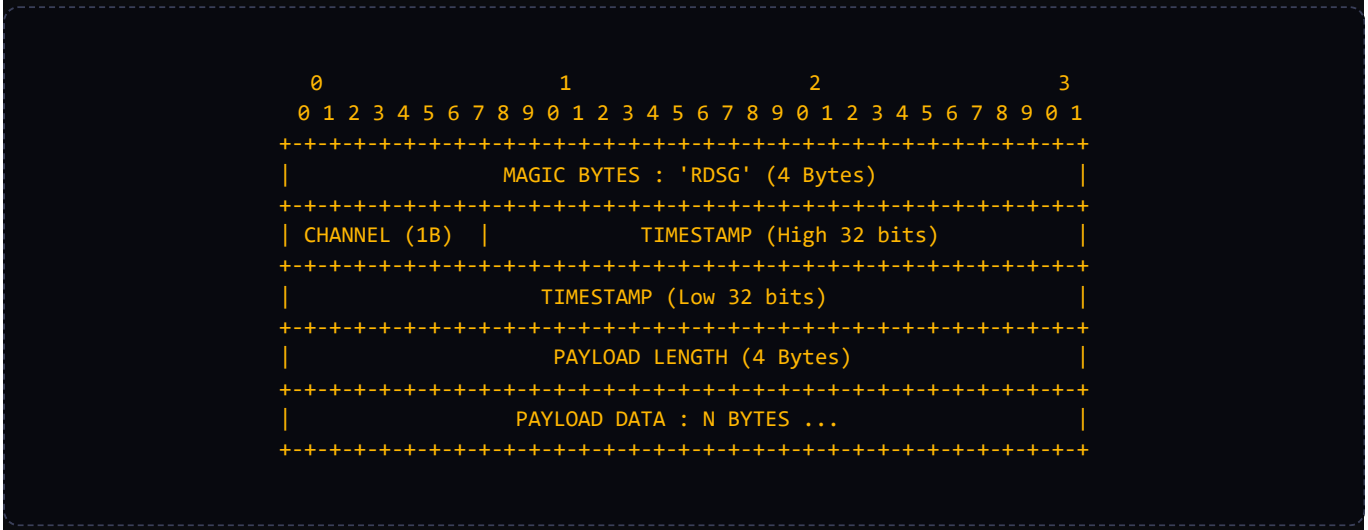
2. **Shannon Entropy H(X) (Measure of Chaos: 0.0 to 8.0)**
   A single key press like 'A' has entropy **0.0**. Normal English sentences score **~3.8 to 4.5**. A compressed stolen password database or ransomware payload is pure chaos, scoring **> 7.6**.

3. **Package Size (Bytes)**
   Keystrokes are 1 to 4 bytes. Clipboard copies are 50 to 2,000 bytes. Giant dumping bursts are uniform chunks of thousands of bytes.

4. **Sliding 5-Second Burst Ratio**
   The bouncer counts how many packets entered the string over the last 5,000 milliseconds. If the frequency exceeds natural limits, quarantine triggers.

3. The 17-Byte Magic Envelope

Every single piece of data flying across Red String is wrapped inside a deterministic, rock-solid 17-byte binary header. No bulky HTTP headers, no JSON packaging waste.



Field Breakdown:

FIELD	SIZE	PURPOSE	VALUES
Magic	4 Bytes	Verification stamp	`'RDSG'` (0x52445347)
Channel ID	1 Byte	Data routing lane	0x01 (Clipboard), 0x02 (Keystroke), 0x03 (File)
Timestamp	8 Bytes (uint64)	Exact epoch milliseconds	Big-Endian standard network byte order
Payload Length	4 Bytes (uint32)	Exact payload buffer size	Bounded by 50 MB safety ceiling
Payload	N Bytes	The opaque secret payload	Encrypted / Raw user bytes

Why Big-Endian ('!')?

Because network standards and the **IBM LinuxONE (s390x) mainframe** are natively Big-Endian (Most Significant Byte first). Red String speaks native mainframe language without endianness translation penalties!

4. The Live Demo Walkthrough (For Judges)

When you run `~/red_string/run_demo.sh`, the demonstration walks through three distinct phases:

I **PHASE I: The Good Monkey (Baseline Normal Sync)**

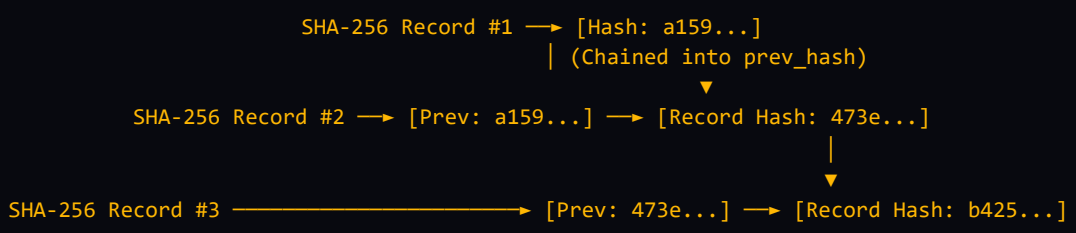
What happens: 15 packets of human keystrokes and clipboard clips are sent over TCP.
The ML Score: +0.28 (Safe Green).
The Verdict: `NOMINAL_ALLOW`.
Result: All text is safely committed to the local memory vault.

II **PHASE II: The Evil Robot (Automated Threat Burst)**

What happens: Malware attacks at 2ms intervals, dumping high-entropy compressed data (entropy 7.7).
The ML Score: Plummets to -0.19 (Critical Red).
The Verdict: `QUARANTINE_SEVER`.
Result: At packet #2, the bouncer cuts the socket. Buffer is wiped. Attacker gets broken pipe. Zero bytes leaked!

III **PHASE III: The Tamper-Proof Audit Notebook**

What happens: The sever event is written into `forensic_audit_trail.jsonl`.
The Math: Every record links to the previous record's SHA-256 hash.
Result: If an attacker tries to delete line 5, all subsequent hashes break! Non-repudiable audit proof for HIPAA and SOC2 compliance.



5. Quick Cheat Sheet & Commands

Everything you need to control Red String on your IBM LinuxONE mainframe instance:

1. Connect to LinuxONE:

```
POWERSHELL (RUN FROM YOUR LAPTOP)
ssh -i "E:\Hackathon edson.pem" linux1@148.100.108.184
```

2. Run the Complete Live Pitch Demo:

```
LINUXONE TERMINAL
~/red_string/run_demo.sh
```

Follow the interactive prompts on screen. Press [ENTER] to advance through Phase 1, Phase 2, and the Forensic Audit Trail.

3. Run the Native High-Performance Rust Server:

```
LINUXONE TERMINAL
~/red_string/run_rust_server.sh
```

Runs the compiled s390x Tokio binary. Listens on `0.0.0.0:7788` and displays the pairing token payload.

4. Run the Edge ML Performance Benchmark:

```
LINUXONE TERMINAL
~/red_string/run_tests.sh
```

Executes all 6 unit tests in 0.05 seconds and proves that inference latency is ~0.61 ms.

Emergency Port Reset:

If you ever see `Address already in use (7788)`, run this one-liner to free the port:

```
sudo fuser -k 7788/tcp
```